

Module 05

RAG Fundamentals

Retrieval-Augmented Generation



Achmad Zaenuri · June 2, 2026

Act 1

Apa itu RAG?

Memberi LLM akses ke pengetahuan eksternal

Masalah LLM

Kenapa LLM saja tidak cukup?

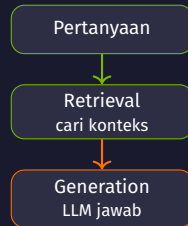
- ▶ LLM menjawab dari **ingatan** hasil pelatihannya saja
 - Masalah 1: **halusinasi** — mengarang jawaban meyakinkan tapi salah
 - Masalah 2: **knowledge cutoff** — tak tahu kejadian setelah training
- ▶ Contoh: tanya berita minggu lalu → LLM bisa salah / menebak
- ▶ Akibatnya: jawaban tak bisa dipercaya untuk fakta terbaru / spesifik

LLM pintar berbahasa, tapi **bukan sumber kebenaran fakta** — ingatannya terbatas dan kadang ngawur.

Solusi: RAG

Retrieval-Augmented Generation

- ▶ **RAG = Retrieval + Generation**
- ▶ **Retrieval:** cari potongan info relevan dari dokumen (knowledge base)
- ▶ **Generation:** LLM menyusun jawaban dari **context** yang ditemukan
- ▶ Hasilnya: jawaban faktual, bisa di-update, halusinasi berkurang



RAG tidak mengubah otak LLM — ia **membekali** LLM dengan konteks faktual segar sebelum menjawab.

Analogi: Ujian Buka Buku

LLM tidak menjawab dari ingatan saja

Tanpa RAG (tutup buku):

- ▶ Jawab dari ingatan → mudah lupa / salah

Dengan RAG (buka buku):

- ▶ Boleh lihat "buku contekan" dulu
- ▶ **Retrieval** = buka halaman relevan dengan soal
- ▶ Konteks dibekalkan ke LLM sebelum menjawab

LLM
+ buku contekan
(knowledge base)

Dibekali konteks faktual dulu, baru menjawab — bukan tebakan dari ingatan.

Act 2

Komponen RAG

Embedding, FAISS, knowledge base, dan generator

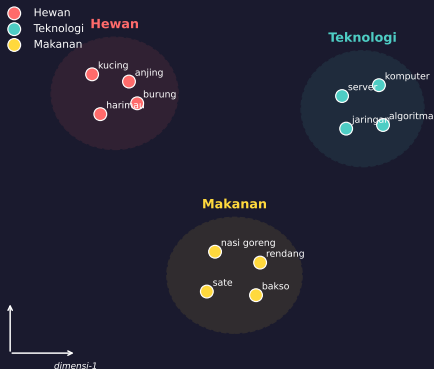
Embeddings & SentenceTransformer

Mengubah teks menjadi angka yang bisa dibandingkan

- ▶ **Embedding:** teks → **vektor** (deretan angka)
- ▶ Kalimat **mirip makna** → vektor **berdekatan**
- ▶ Komputer paham jarak antar-angka, bukan kata
- ▶ Model **all-MiniLM-L6-v2** via **SentenceTransformer**

```
1 embedder = SentenceTransformer(  
2     'sentence-transformers/all-MiniLM-L6  
   -v2')  
3 v = embedder.encode(["ibu kota Prancis  
   "])
```

Ruang Embedding: Makna Mirip -> Berdekatan



Similarity Search & FAISS

Mencari vektor terdekat dengan cepat

- ▶ **FAISS** (Facebook AI Similarity Search): pencari vektor mirip super cepat
- ▶ **IndexFlatL2**: ukur **jarak Euclidean (L2)** antar vektor
- ▶ Makin kecil jarak → makin mirip maknanya
- ▶ Ambil **top-k** terdekat (notebook k=1)
- ▶ Tetap cepat walau knowledge base ribuan dokumen

```
1 index = faiss.IndexFlatL2(  
2     embedding_size)  
3 distances, indices = index.search(  
4     query_embedding.astype('float32')  
5     , k)
```

FAISS = mesin pencari makna: dari query ke dokumen paling relevan dalam sekejap.

Knowledge Base

Sumber fakta yang dibaca sistem RAG

- ▶ **Knowledge base:** kumpulan dokumen / fakta sumber jawaban
- ▶ Contoh: Paris, Python, Machine Learning, Tembok China, dll
- ▶ Tiap dokumen di-encode() jadi vektor
- ▶ Semua vektor masuk **FAISS** via `index.add()`
- ▶ Tambah dokumen = encode lalu masukkan (tanpa latih ulang)

```
1 knowledge_base = [  
2     "...Paris...", "...Python...", ...]  
3 embeddings = embedder.encode(  
4     knowledge_base)  
5 index.add(embeddings.astype('float32'  
    ))
```

Knowledge base = ingatan eksternal LLM: cukup tambah dokumen, tak perlu latih ulang.

Generator LLM

TinyLlama menyusun jawaban akhir

- ▶ **Generator:** LLM penulis jawaban — **TinyLlama 1.1B Chat**
- ▶ `torch.float32` (presisi penuh) agar **stabil di CPU**
- ▶ Dibungkus `pipeline("text-generation")`
- ▶ `temperature=0.3` → fokus; makin tinggi makin acak
- ▶ `top_p=0.9` → **nucleus sampling**

```
1 generator = pipeline(  
2     "text-generation",  
3     model=model, tokenizer=tokenizer,  
4     max_length=512, do_sample=True,  
5     temperature=0.3, top_p=0.9)
```

TinyLlama float32 + sampling rendah: jawaban fokus berbasis konteks, stabil tanpa GPU.

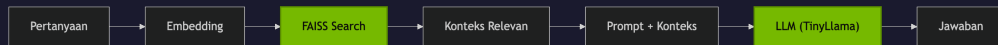
Act 3

Pipeline RAG

Retrieve, Augment, Generate

Alur End-to-End

Tiga langkah dari pertanyaan ke jawaban



Retrieve → **Augment** → **Generate**: LLM dibekali konteks faktual dulu (fungsi `ask_question()`).

Langkah 1 — Retrieve

Mencari konteks paling relevan

- ▶ Query di-**embed** jadi **vektor**
- ▶ Dicocokkan ke **FAISS** (IndexFlatL2)
- ▶ Ambil **top-k** terdekat (k=1)
- ▶ Teks asli diambil dari `knowledge_base`

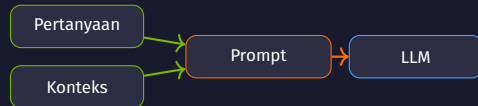
```
1 def retrieve_relevant_context(query, k=1):
2     q_emb = embedder.encode([query])
3     dist, idx = index.search(
4         q_emb.astype('float32'), k)
5     docs = [knowledge_base[i] for i in idx[0]]
6     return "\n".join(docs)
```

Makin dekat letak vektor, makin mirip maknanya — FAISS cari tetangga terdekat dengan cepat.

Langkah 2 — Augment

Menyisipkan konteks ke dalam prompt

- ▶ **Augment:** per kaya **prompt** dengan konteks hasil retrieve
- ▶ Konteks + pertanyaan asli digabung jadi satu prompt
- ▶ Format: bagian `Context:` lalu `Question:`
- ▶ LLM punya bahan faktual, bukan sekadar menebak



Augment = jembatan antara **retrieve** dan **generate**: konteks + pertanyaan → satu prompt.

Langkah 3 — Generate

LLM menjawab berbasis konteks

- ▶ **Generation:** LLM susun jawaban dari prompt yang diperkaya
- ▶ **TinyLlama** baca konteks → tulis jawaban relevan
- ▶ `float32` (bukan `float16`) agar stabil di CPU
- ▶ `temperature=0.3` → fokus, kurangi **halusinasi**

```
1 response = generator(prompt,  
2     max_length=512, do_sample=True,  
3     temperature=0.3,  
4     num_return_sequences=1)
```

Berbekal konteks faktual, jawaban LLM lebih akurat & tidak mengarang — inti nilai RAG.

Act 4

Praktik & Lanjutan

Kode, batasan, dan langkah berikutnya

Kode Sistem RAG

Dua fungsi inti (disederhanakan): ambil konteks lalu susun jawaban

```
1 def retrieve_relevant_context(query, k=1):
2     q_emb = embedder.encode([query])
3     dist, idx = index.search(q_emb.astype('float32'), k)
4     return "\n".join(knowledge_base[i] for i in idx[0])
5
6 def generate_response(query, context):
7     prompt = build_tinyllama_prompt(context, query)    # Context: ... Question:
8     ...
9     out = generator(prompt)[0]['generated_text']
10    return clean_response(out)
11
12 # ask_question(): retrieve -> augment prompt -> generate
```

Pola RAG: **retrieve** → **augment** prompt → **generate**. (Notebook asli pakai chat template TinyLlama.)

Tips & Batasan

Kualitas jawaban bergantung pada beberapa pilihan

- ✓ **Ukuran chunk:** terlalu besar → konteks kabur; terlalu kecil → info penting hilang
- ✓ **Nilai k :** berapa konteks diambil; notebook $k=1$ (fokus), kadang butuh lebih
- ✓ **Kualitas embedding:** model (all-MiniLM-L6-v2) menentukan ketepatan dokumen relevan
- **RAG bukan jaminan 100% benar:** konteks kurang relevan → jawaban tetap meleset

RAG **mengurangi** halusinasi, bukan menghapusnya. Jawaban hanya sebaik konteks yang berhasil diambil.

Ringkasan & Lanjutan

Empat komponen RAG dan jembatan ke Module o6

Komponen	Library / Model	Tujuan
Embedding	SentenceTransformer (all-MiniLM-L6-v2)	Ubah teks jadi vektor
Index	FAISS (IndexFlatL2)	Cari vektor terdekat cepat
Generator	TinyLlama-1.1B-Chat (float32)	Susun jawaban akhir
Pipeline	Gabungan 1-3	Retrieve → augment → generate

Module o6: RAG yang sama dipercepat GPU NVIDIA (faiss-gpu + presisi float16).

Terima Kasih!

Pertanyaan? Diskusi?

RAG Fundamentals · Modul 05
Navasena Gen-ML Course